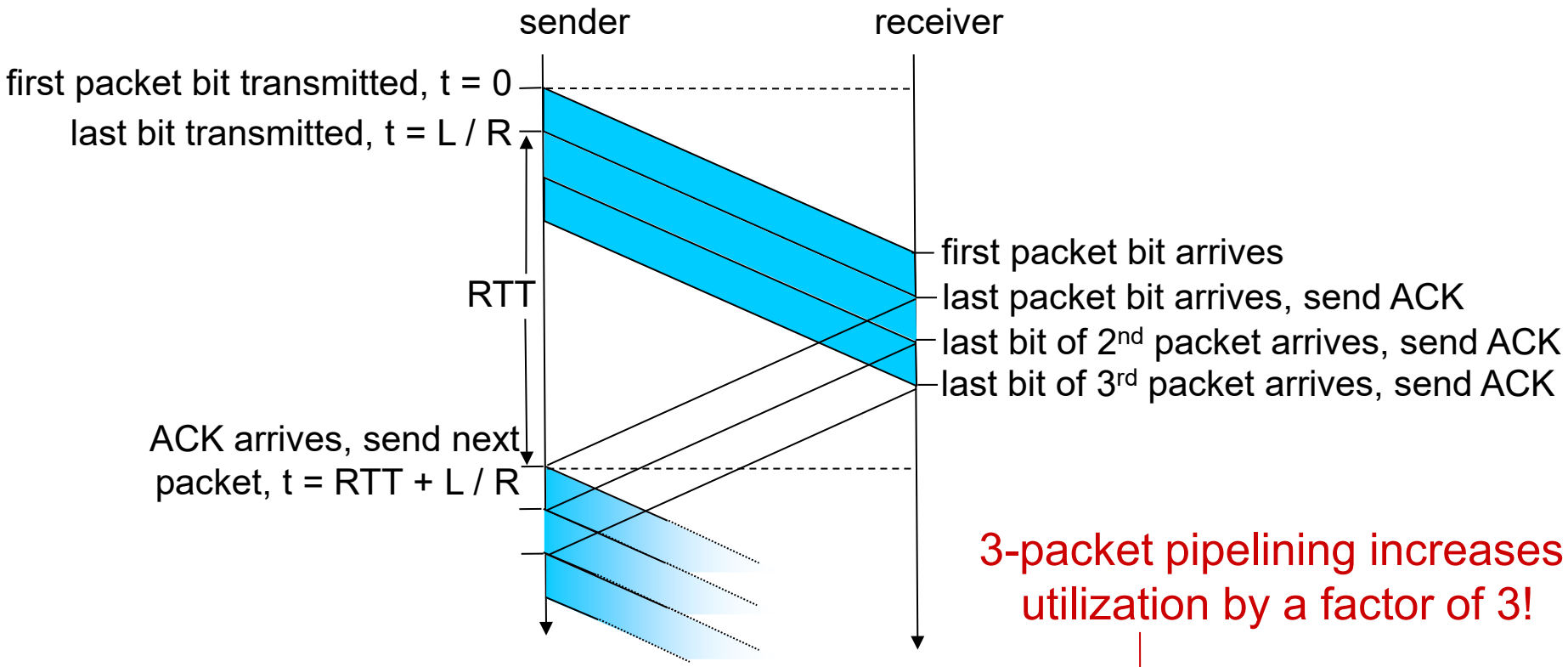
Help: Implementing a Reliable Transport Protocol

Outline :

- Recap of related concepts (pg 2-6)
  - Pipelining (sending a window of packets) (pg3)
  - GBN's window in operation at sender and receiver (pgs 4,5)
  - GBN in action one case (pg5)
  - TCP segment as an example for the GBN packet data structure (pg6)
- How the routines to be implemented map to the FSMs
  - Sender (pg7)
  - Receiver (pg8)
  - Consider the two pages be the most important
- Suggestions with steps for code development and testing (pg9)

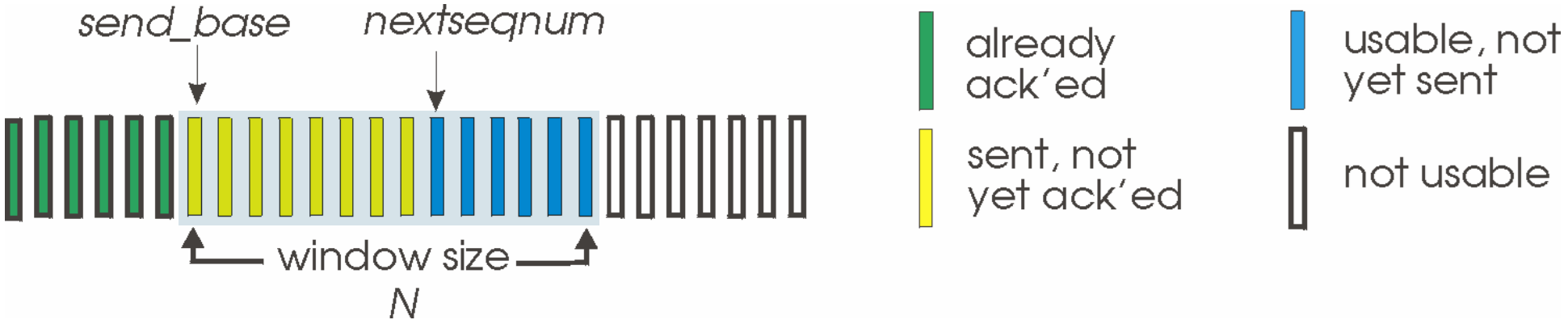
Pipelining: increased utilization



3-packet pipelining increases utilization by a factor of 3!

Go-Back-N: sender

- sender: “window” of up to N, consecutive transmitted but unACKed pkts
  - k-bit seq # in pkt header

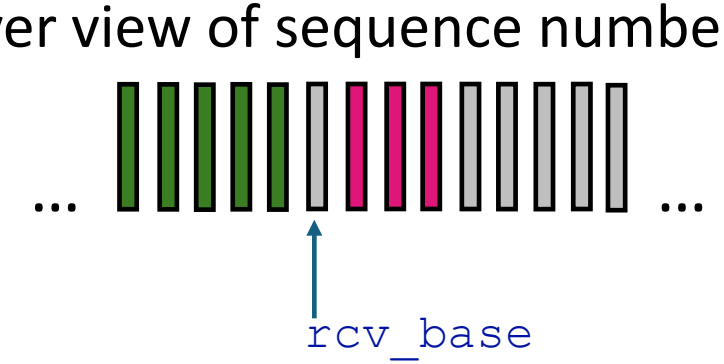


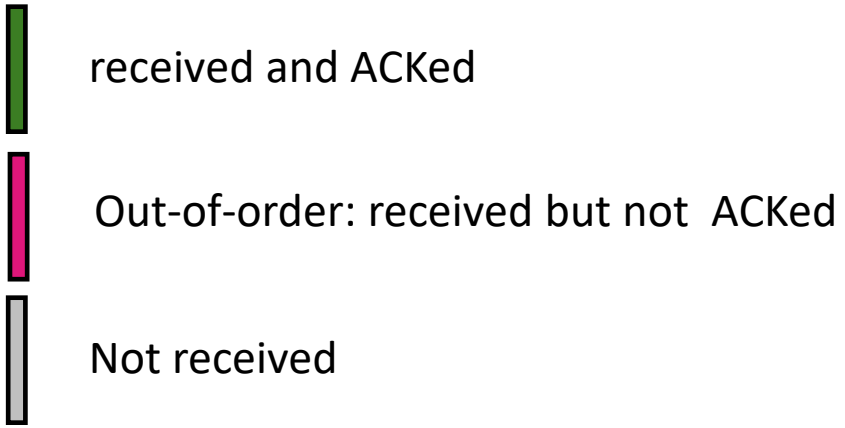
- *cumulative ACK:* ACK(*n*): ACKs all packets up to, including seq # *n*
  - on receiving ACK(*n*): move window forward to begin at *n+1*
- timer for oldest in-flight packet
- *timeout(n):* retransmit packet n and all higher seq # packets in window

Go-Back-N: receiver

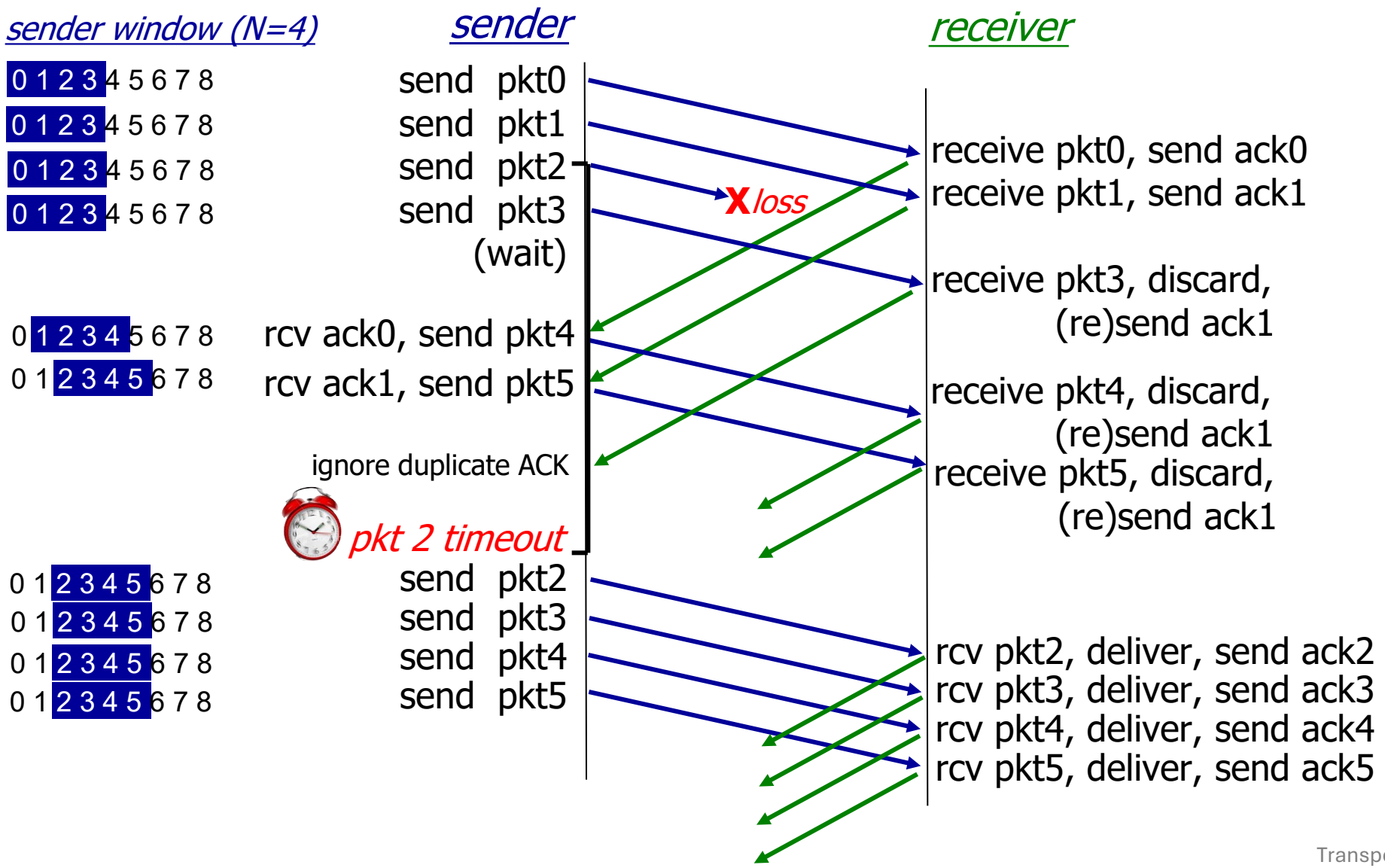
- ACK-only: always send ACK for correctly-received packet so far, with highest *in-order* seq #
  - may generate duplicate ACKs
  - need only remember `rcv_base`
- on receipt of out-of-order packet:
  - can discard (don't buffer) or buffer: an implementation decision
  - re-ACK pkt with highest in-order seq #

Receiver view of sequence number space:

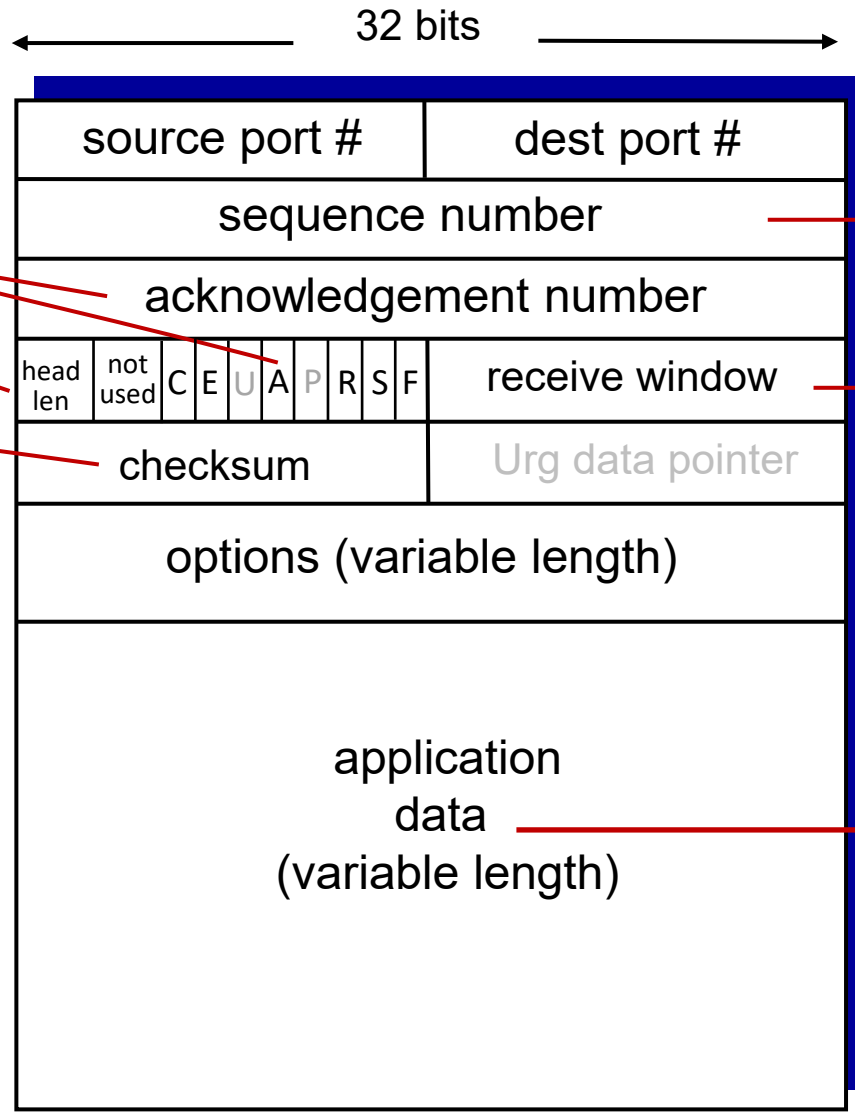




Go-Back-N in action



TCP segment structure



ACK: seq # of next expected byte; A bit: this is an ACK

length (of TCP header)

Internet checksum

segment seq #: counting bytes of data into bytestream (not segments!)

flow control: # bytes receiver willing to accept

data sent by application into TCP socket

```
struct pkt {
  int seqnum;
  int acknum;
  int checksum;
  char payload[20];
 };
```

payload is the msg

```
struct msg {
  char data[20];
 };
```

GBN: sender extended FSM

A_output(message) struct msg message;

{add headers to make pkt, call tolayer3; handle seq, timer } // called in main via event From_Layer5, set by main

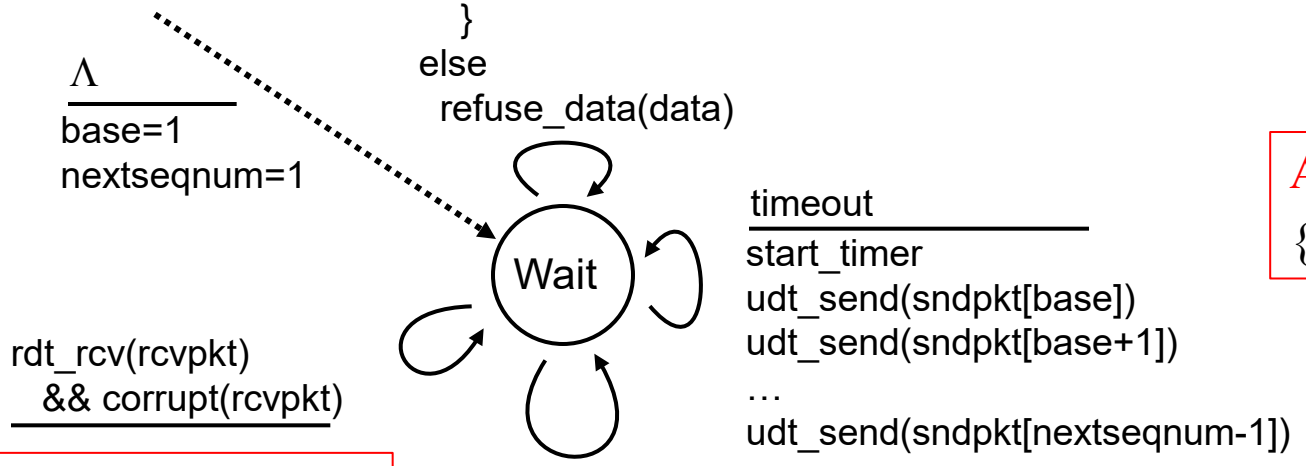
A_init() // main

{base, nextseq}

A_input (packet) struct pkt packet;

{checksum, retx upon timeout, or update base, timer }

// called in main via event From_ Layer3, set by tolayer3



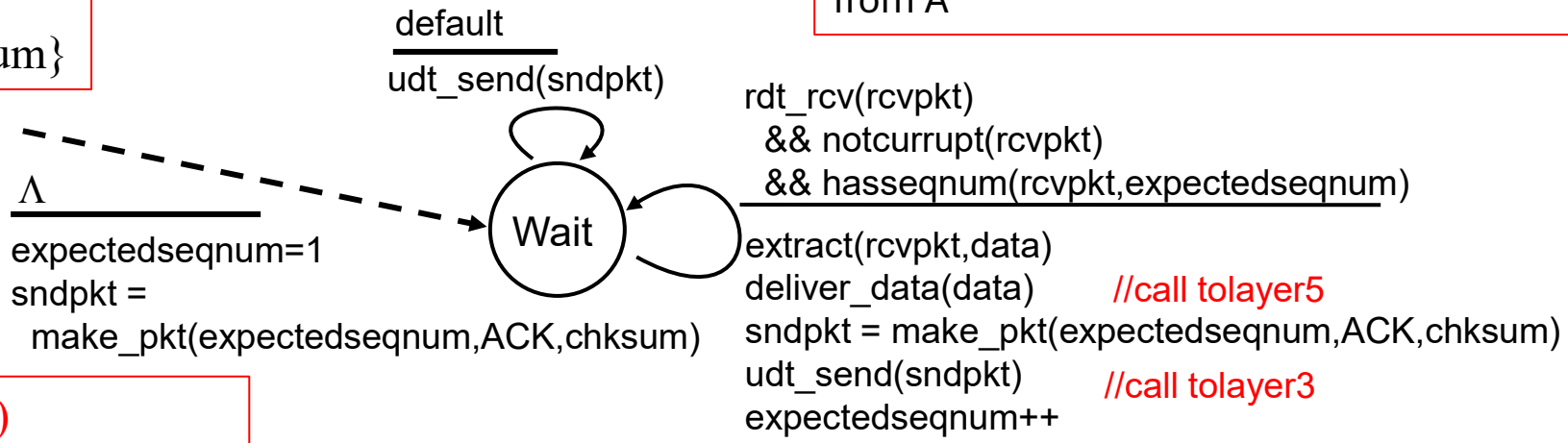
A_timerinterrupt ()

{timer, tolayer3(AorB,pkt)}

GBN: receiver extended FSM

B_input (packet) struct pkt packet;
{checksum, toLayer5, or send ack, update eSeq. }
// called in main via event From_ Layer3, set by tolayer3 from A

B_init() //
{expectedseqnum}



B_timerinterrupt ()
{timer, tolayer3(AorB,pkt)}

ACK-only: always send ACK for correctly-received pkt with highest *in-order* seq #

- may generate duplicate ACKs
- need only remember **`expectedseqnum`**

- out-of-order pkt:
  - discard (don't buffer): *no receiver buffering!*
  - re-ACK pkt with highest in-order seq #

Suggested development and testing steps

- Start with:
  - Code for window, send/receive/sending ack
  - Test with a small window, reliable channel
- Add code handling error
  - Testing with error
- Add code handling loss
  - Testing with loss
- Test with combined condition for error, loss, etc.